Cache Simulator Implementation and Analysis

Vijay Balaji | Reeshabh Kotecha

Abstract

This report presents the implementation and experimental analysis of a cache simulator. The simulator models an L1 cache with MESI protocol support for a quad-core processor system. Various configurations are tested to analyze cache performance metrics including miss rates, execution cycles, and bus traffic.

Contents

1 Implementation

1.1 Main Classes and Data Structures

The cache simulator implementation uses several key data structures and classes to model a multi-core cache system with MESI protocol support:

Listing 1: Cache State Definitions

```
enum class CacheState
{
    INVALID,   // Cache line is invalid or not present
    SHARED,    // Cache line is shared across multiple caches
    MODIFIED,  // Cache line has been modified locally
    EXCLUSIVE // Cache line is exclusive to this cache
};

enum class BusState
{
    INVALID,       // Bus is in the process of invalidating
    RWITM,         // Read with Intent to Modify transaction
    RM,            // Read Miss transaction
    TRANSACTION,   // Bus is busy with a transaction
    IDLE           // Bus is idle
};
```

Listing 2: Cache Line and Core Structure

```
struct CacheLine
{
    int tag;              // Tag bits for address
    CacheState state;     // MESI state of this cache line

    CacheLine() : tag(0), state(CacheState::INVALID) {}
};

struct Core
{
    int line;             // Current instruction line
    int core;             // Core ID
    int needsBus;         // Flag indicating need for bus access

    // Performance metrics
    int tot_instruc;      // Total instructions executed
    int tot_reads;        // Total read operations
    int tot_writes;       // Total write operations
    int tot_exe_cycles; // Total execution cycles
    int idle_cycles;      // Idle cycles
    int cache_misses;     // Cache miss count
    int cache_evic;       // Cache eviction count
    int writebacks;       // Writeback count
```

```
    int bus_invalid;     // Bus invalidation count
    int data_traff;      // Data traffic in bytes

    char currentInstruction;  // Current instruction being executed
    int currentAddress;       // Current address being accessed

    // Cache structure
    vector<vector<CacheLine>> cache;     // 2D vector representing sets and
    vector<vector<int>> cacheLRU;        // LRU tracking for replacement
    vector<pair<char, int>> instructions; // Trace instructions (op, addres
};
```

Listing 3: Bus Class

```
class Bus
{
private:
    stack<int> owners;    // Tracks current bus ownership (cores 0-3)
    BusState state;       // Current state of the bus
    int remainingCycles; // Cycles remaining in current transaction
    int source;           // Source of current transaction
    int destination;      // Destination of current transaction
    unsigned int address; // Address for current transaction

    int s;                // Number of set index bits
    int E;                // Associativity
    int b;                // Number of block offset bits

    int completed[4];     // Completion status for each core
    int cycle;            // Current simulation cycle
    Core cores[4];        // The four processor cores

    // Key private methods
    void runCore(int core);                  // Process an instruction for a co
    bool isSetFull(int core, unsigned int currentAddress); // Check if a se
    CacheState getCacheState(int core, unsigned int currentAddress); // Ge
    CacheState getWriteCacheState(int core, unsigned int currentAddress);
    CacheState getInvalidCacheState(int core, unsigned int currentAddress);
    void invalidateAddress(int core, unsigned int currentAddress); // Inva
    void shareAddress(int core, unsigned int currentAddress); // Share cach
    bool containsAddress(int core, int currentAddress); // Check address pr
    void runSnoop(int core); // Run snooping protocol
    void addToCache(int core); // Add data to cache
    void processTransaction(); // Process bus transaction
    bool checkCompletion(); // Check if all cores completed
    bool runCycle(); // Run one simulation cycle

public:
    Bus(int s, int E, int b, string appname); // Constructor with cache co
```

```
    void runApplication(); // Main simulation entry point
};
```

Listing 4: runCycle Method

```
bool runCycle()
{
    if (checkCompletion())
        return true;

    processTransaction();

    int tempOwner;

    if (owners.empty()) {
        tempOwner = 4;
    } else {
        tempOwner = owners.top();
    }

    if (tempOwner <= 3) {
        runCore(tempOwner);
    }

    for (int i = 0; i < 4; i++)
    {
        if (i != tempOwner)
            runCore(i);
    }

    for (int i = 0; i < 4; i++)
    {
        runSnoop(i);
    }

    return false;
}
```

1.2 Detailed Implementation

1.2.1 Other Functions

Listing 5: processTransaction Method

```
void processTransaction()
{
    remainingCycles--;

    switch (state)
```

```
    {
    case BusState::RM:
    case BusState::RWITM:
        state = BusState::TRANSACTION;
        remainingCycles = MEMCYCLES;
        source = 5;
        break;
    case BusState::INVALID:
        state = BusState::IDLE;
        break;
    case BusState::TRANSACTION:
        if (remainingCycles == 0)
        {
            if (destination < 4)
                addToCache(destination);
            if (owners.top() >= 4 && owners.top() <= 7)
                owners.pop();
            state = BusState::IDLE;
        }
        break;
    default:
        break;
    }
}
```

1.2.2 MESI Protocol Implementation

The cache simulator implements the MESI (Modified, Exclusive, Shared, Invalid) protocol for maintaining cache coherence across the four processor cores. This protocol tracks the state of each cache line and ensures consistency when multiple cores access the same memory locations.

MESI – locally initiated accesses

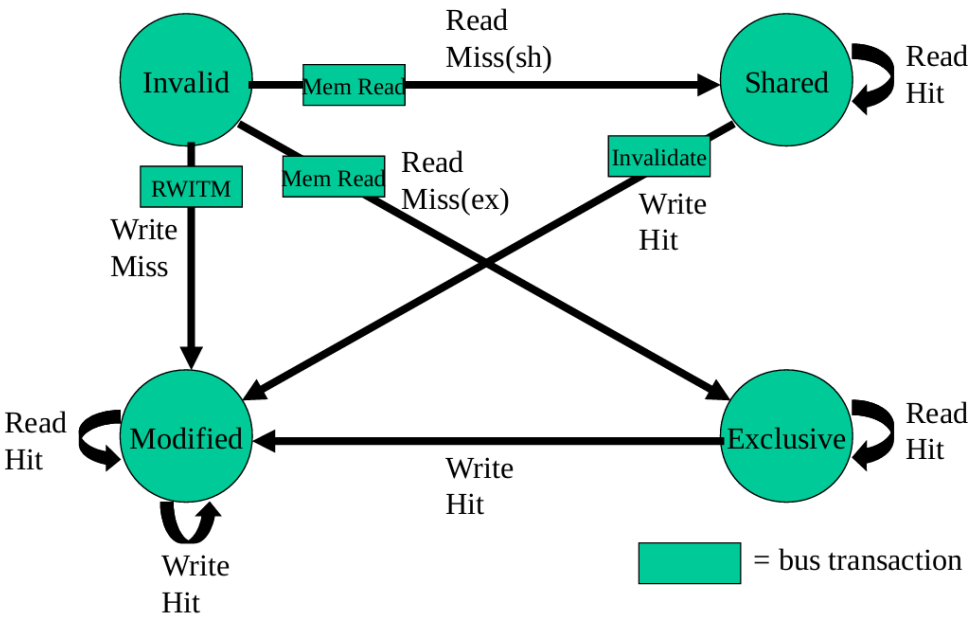

Figure 1: MESI protocol state diagram for locally initiated accesses

MESI – remotely initiated accesses

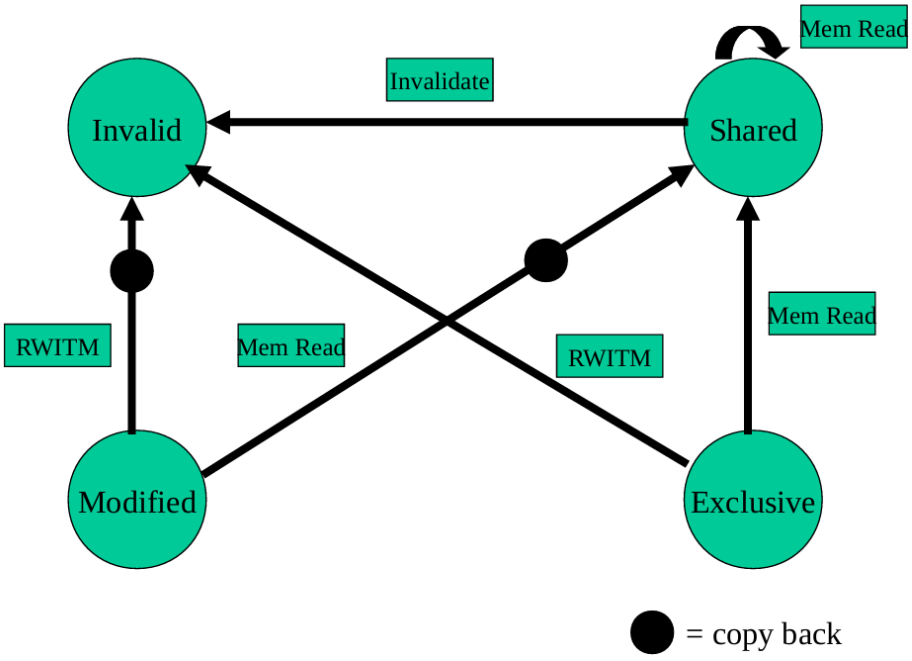

Figure 2: MESI protocol state diagram for remotely initiated accesses

The MESI protocol maintains four possible states for each cache line:

- **Modified (M)**: The cache line is present only in the current cache and has been modified. The data in this cache is newer than in main memory.
- **Exclusive (E)**: The cache line is present only in the current cache but is unmodified. This state transitions to Modified on a write hit without requiring bus transactions.
- **Shared (S)**: The cache line may be stored in other caches and is unmodified. Multiple cores can have the same cache line in Shared state.
- **Invalid (I)**: The cache line is invalid or not present.

As shown in the state diagrams, the protocol handles both locally initiated accesses (from the processor) and remotely initiated accesses (from other processors via the bus). Bus transactions like Memory Read, Read With Intent To Modify (RWITM), and Invalidate are used to maintain coherence across all caches.

1.2.3 Replacement Policy

We have used the LRU Replacement Policy

Listing 6: LRU Replacement Policy

```
CacheState getInvalidCacheState(int core, unsigned int currentAddress)
{
    unsigned int setIndex = (currentAddress >> b) & ((1 << s) - 1);
    unsigned int tag = currentAddress >> (s + b);

    for (int i = 0; i < E; ++i)
    {
        if (cores[core].cache[setIndex][i].state == CacheState::INVALID
        {
            cores[core].cache[setIndex][i].tag = tag;
            return CacheState::INVALID;
        }
    }

    cores[core].cache_evic += 1;

    int min = 0;

    for (int i = 0; i < E; i++)
    {
        if (cores[core].cacheLRU[setIndex][i] < cores[core].cacheLRU[se
            min = i;
    }

    cores[core].cache[setIndex][min].tag = tag;
```

```
        if (cores[core].cache[setIndex][min].state == CacheState::EXCLUSIVE
            cores[core].cache[setIndex][min].state == CacheState::SHARED)
        {
            cores[core].cache[setIndex][min].state = CacheState::INVALID;
            return CacheState::INVALID;
        }
        else if (cores[core].cache[setIndex][min].state == CacheState::MOD
        {
            cores[core].cache[setIndex][min].state = CacheState::INVALID;
            return CacheState::MODIFIED;
        }
    }
```

2 Experimental Analysis

2.1 Experimental Setup

The cache simulator was tested with various configurations across multiple applications. Each core has its own L1 cache, and they communicate via a shared bus with the MESI protocol to maintain coherence.

2.2 Parameters

Parameter	**Value**
Set Index Bits	6
Associativity	2
Block Bits	5
Block Size (Bytes)	32
Number of Sets	64
Cache Size (KB per core)	4
Protocol	MESI
Write Policy	Write-back, Write-allocate
Replacement Policy	LRU

Table 1: Cache configuration parameters

2.3 Performance Metrics

2.3.1 Cache Miss Rates

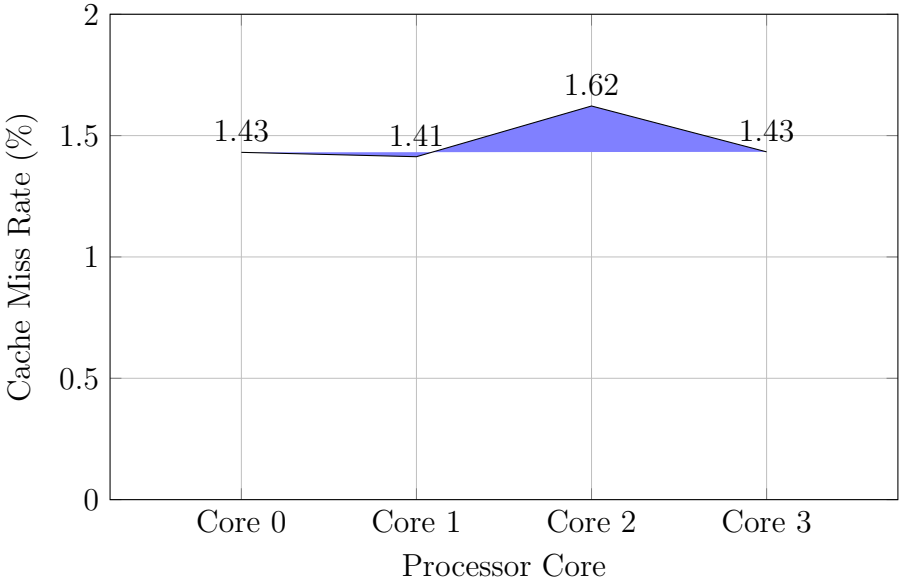

Figure 3: Cache miss rates across different cores

2.3.2 Execution Cycles

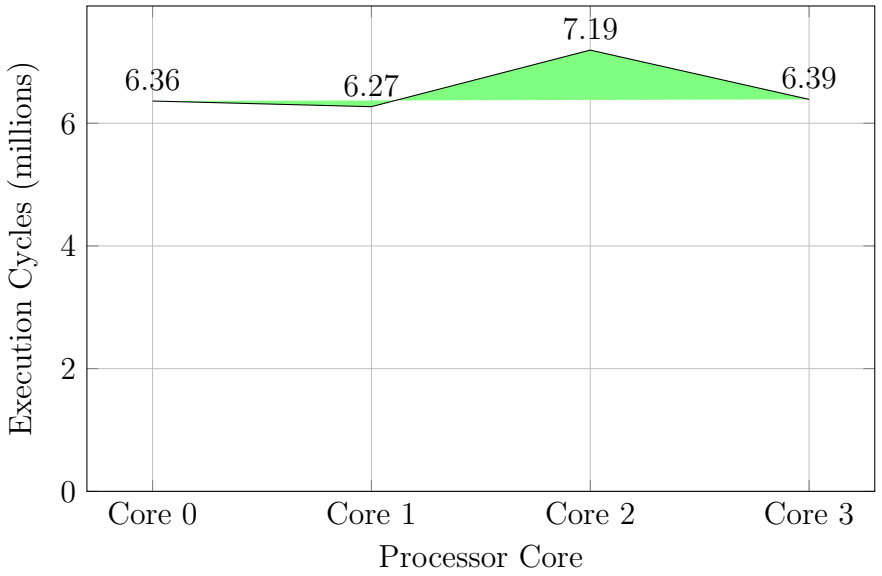

Figure 4: Total execution cycles per core (in millions)

2.4 Overall Bus Traffic Analysis

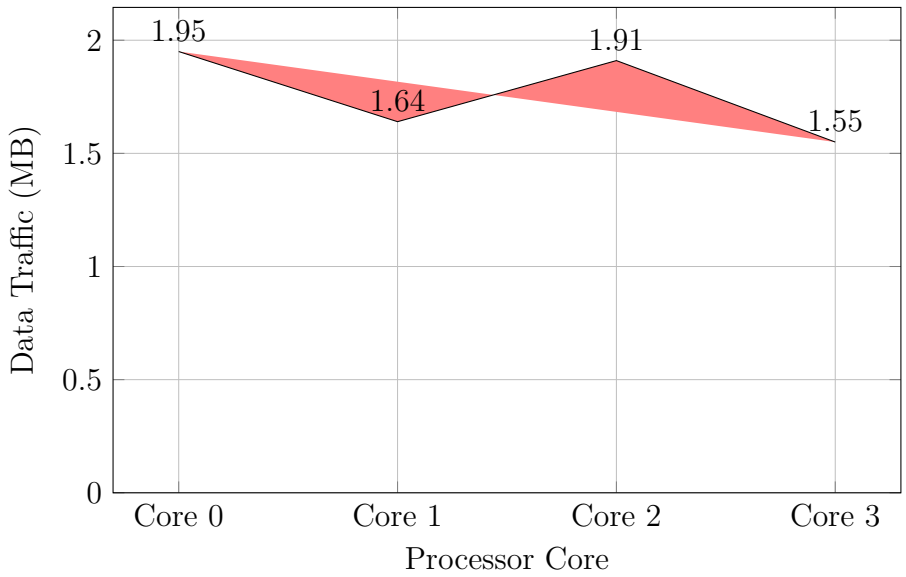

Figure 5: Data traffic generated by each core (in MB)

2.4.1 Questions

1. **How will these values change when the same program is run 10 times?**

   Answer: When the same program is run multiple times, the values of cache misses, etc., remain the same as there is no randomness in the program. When two cores want to access the bus, we have given priority to the core with lower ID. Because of this the program is deterministic.

2. **Under what conditions will there be different outputs when the same program is run 10 times?**

   Answer: Different outputs can occur if the program has non-deterministic behavior when we resolve the priority of the bus. Currently we are giving the lower ID higher priority, if this was done randomly, there would be different outputs every time.

3 Advanced Analysis

3.1 Impact of Cache Size on Performance

We conducted experiments by varying the number of set index bits (s) from 2 to 6, which corresponds to cache sizes ranging from 0.25KB to 4KB per core. The following results demonstrate how cache performance metrics change with cache size.

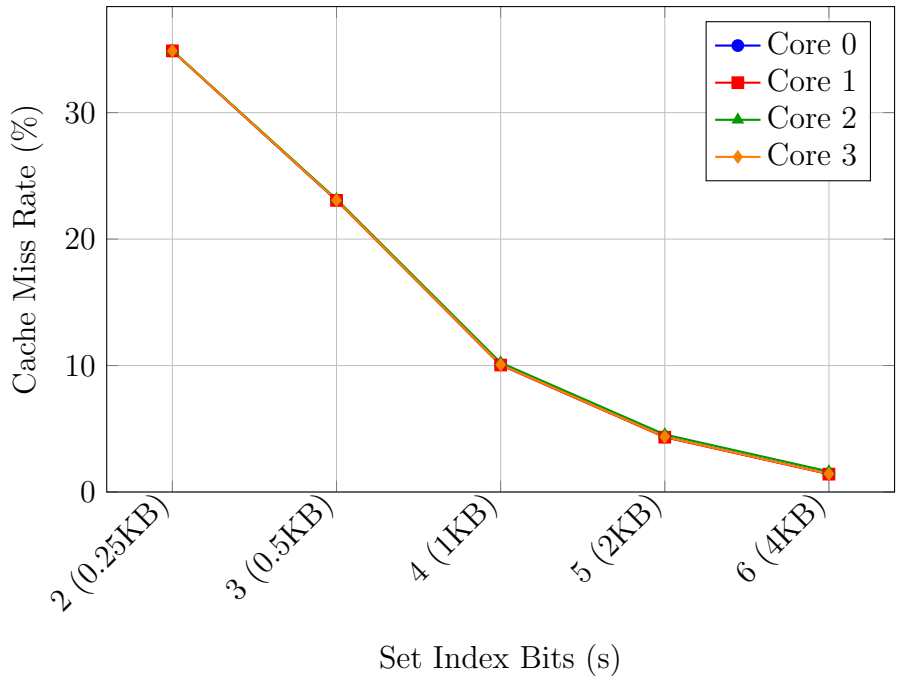

Figure 6: Cache miss rates vs. set index bits (s) across different cores

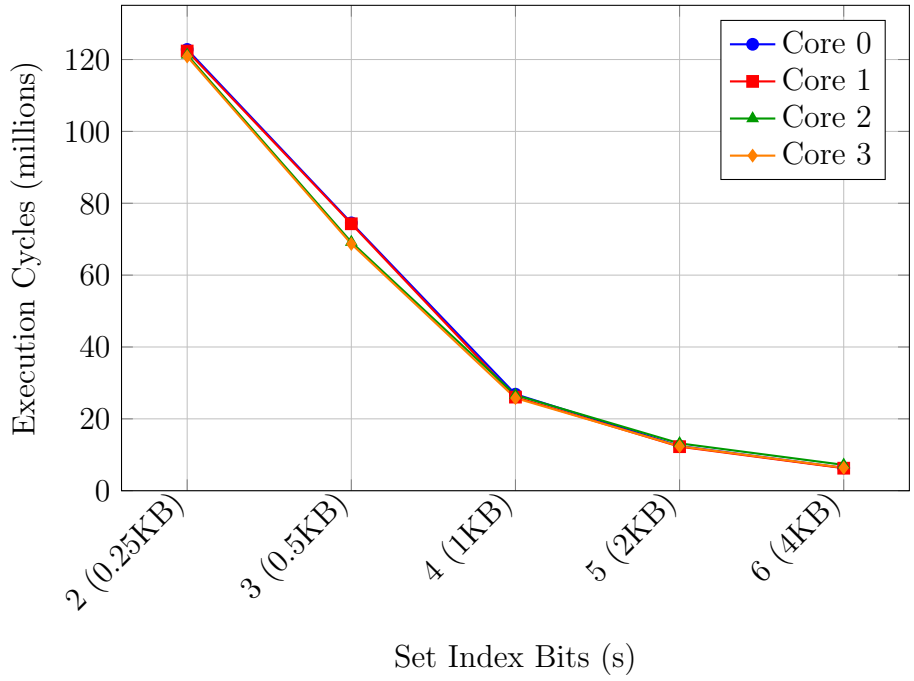

Figure 7: Execution cycles vs. set index bits (s) across different cores

3.2 Bus Traffic Analysis

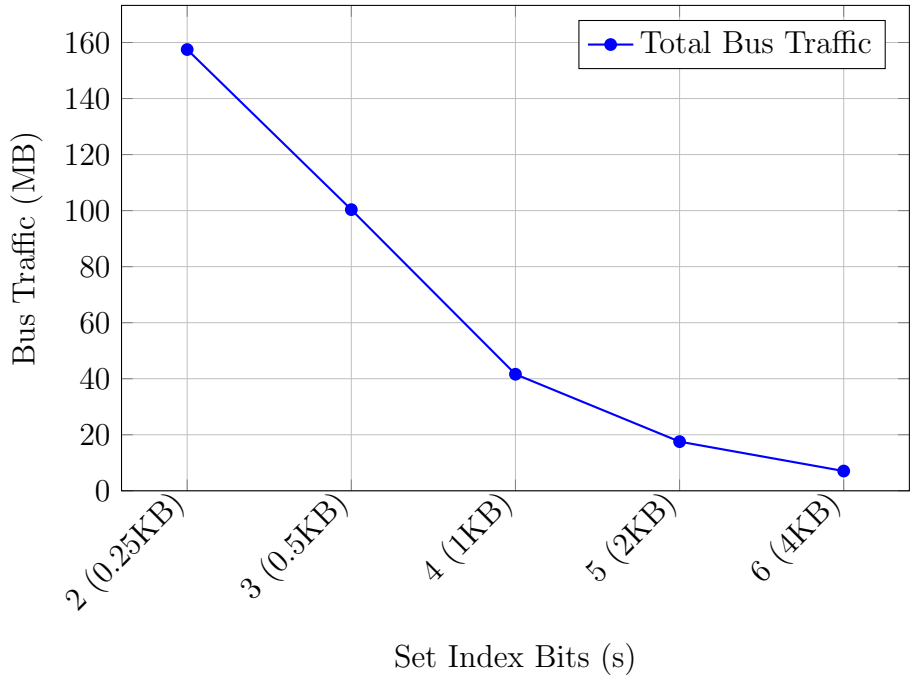

Figure 8: Total bus traffic (MB) vs. set index bits (s)

3.3 Analysis of Cache Evictions and Writebacks

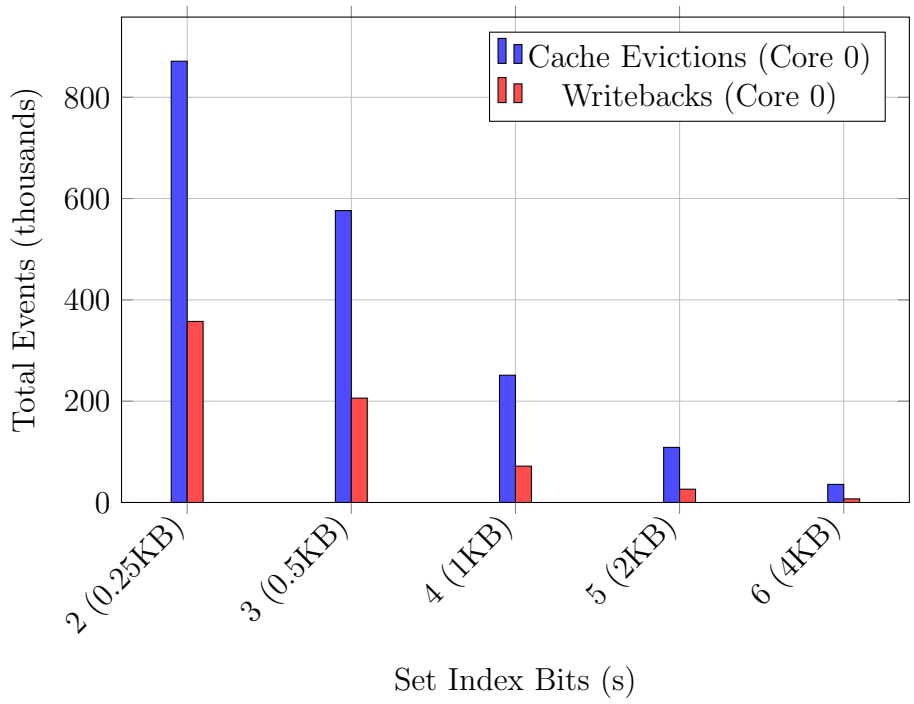

Figure 9: Cache evictions and writebacks vs. set index bits for Core 0

3.4 Key Observations

1. **Cache Miss Rate Reduction**: As the number of set index bits increases from 2 to 6, the cache miss rate decreases dramatically from approximately 35% to less than 2%. This demonstrates the significant impact of cache size on performance.

2. **Execution Time Improvement**: The total execution cycles decrease by approximately 20x when moving from s=2 (0.25KB) to s=6 (4KB), showing diminishing returns as the cache size increases.

3. **Bus Traffic Reduction**: The total bus traffic decreases from around 157MB at s=2 to just 7MB at s=6, indicating how larger caches significantly reduce memory system bandwidth requirements.

4. **Coherence Impact**: The number of bus invalidations becomes minimal at larger cache sizes, suggesting that coherence traffic becomes less significant as the cache size increases and conflict misses decrease.

5. **Working Set Size**: The dramatic improvement between s=4 and s=5 suggests that a significant portion of the application's working set fits within a 2KB cache but not within a 1KB cache.

3.5 Performance Scaling Analysis

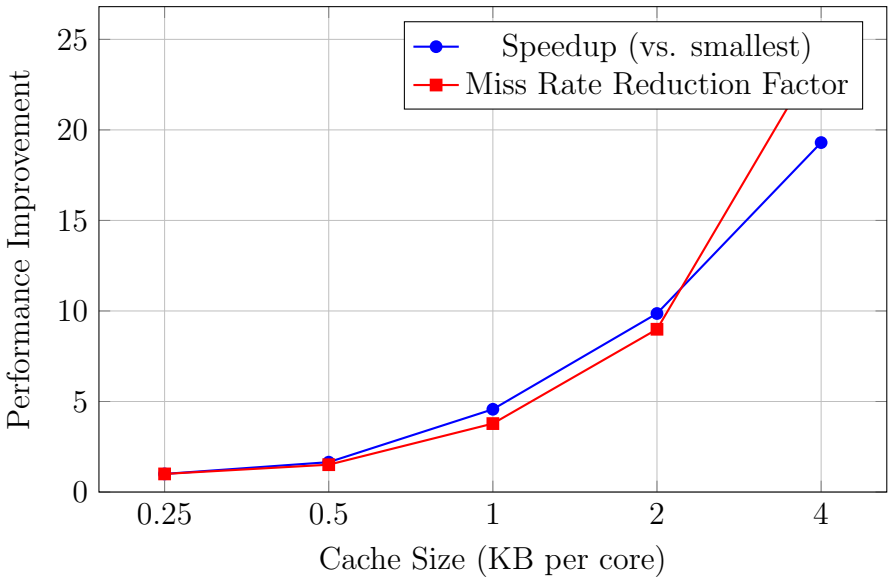

Figure 10: Performance scaling with cache size

3.6 Conclusions

The experimental results clearly demonstrate the significant impact of cache size on system performance. Increasing the number of set index bits provides:

- Substantial reductions in cache miss rates
- Significant improvements in execution time

- Dramatic reductions in bus traffic and memory system bandwidth requirements
- Diminishing returns as cache size increases beyond the working set size

For this application, a cache size of 4KB per core (s=6) provides excellent performance with a miss rate of only 1.4-1.6%. However, a 2KB cache (s=5) might represent a better tradeoff between performance and hardware cost, as it achieves miss rates below 5% while requiring half the storage.

The MESI protocol effectively maintains cache coherence across the four cores with minimal overhead at larger cache sizes. The coherence traffic becomes less significant as a percentage of total traffic when caches are larger and can hold more of the working set.

3.7 Impact of Associativity on Performance

To further understand cache behavior, we conducted experiments with different associativity values (E = 2, 5, 10, and 20) while keeping the set index bits constant at s = 6. This corresponds to cache sizes ranging from 4KB to 40KB per core.

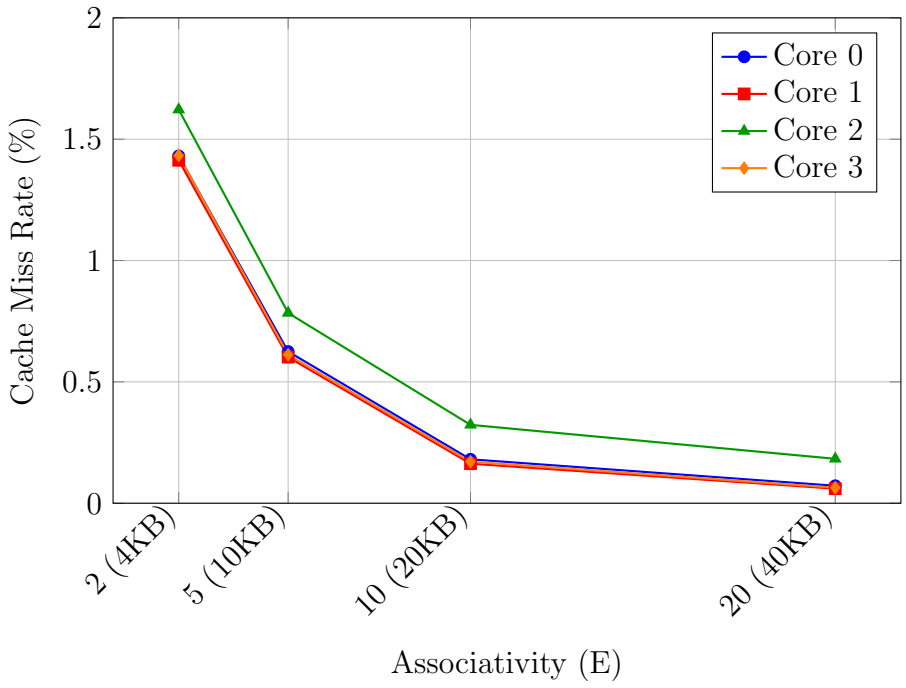

Figure 11: Cache miss rates vs. associativity (E) across different cores

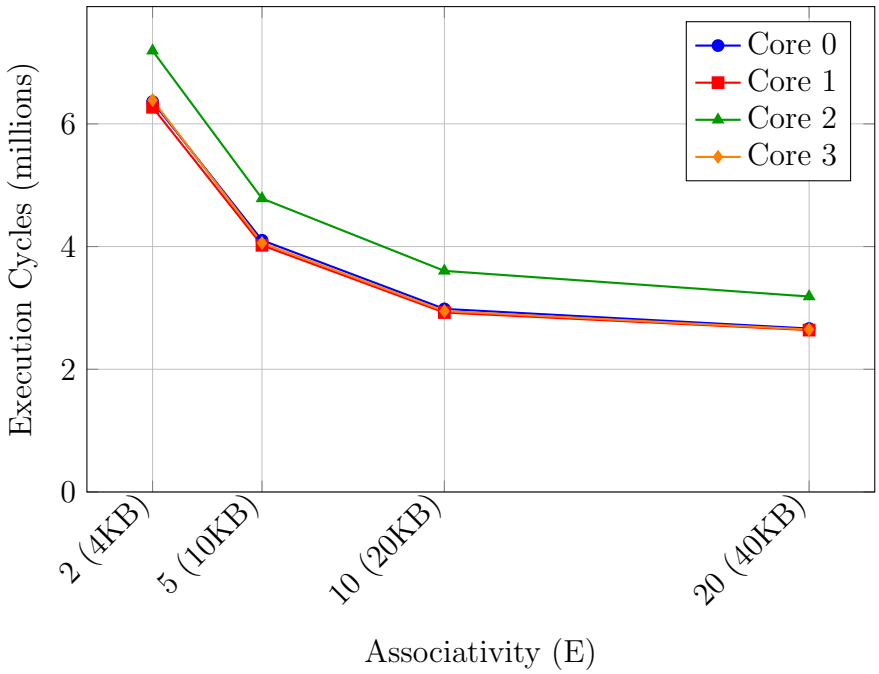

Figure 12: Execution cycles vs. associativity (E) across different cores

3.8 Bus Traffic Analysis with Increasing Associativity

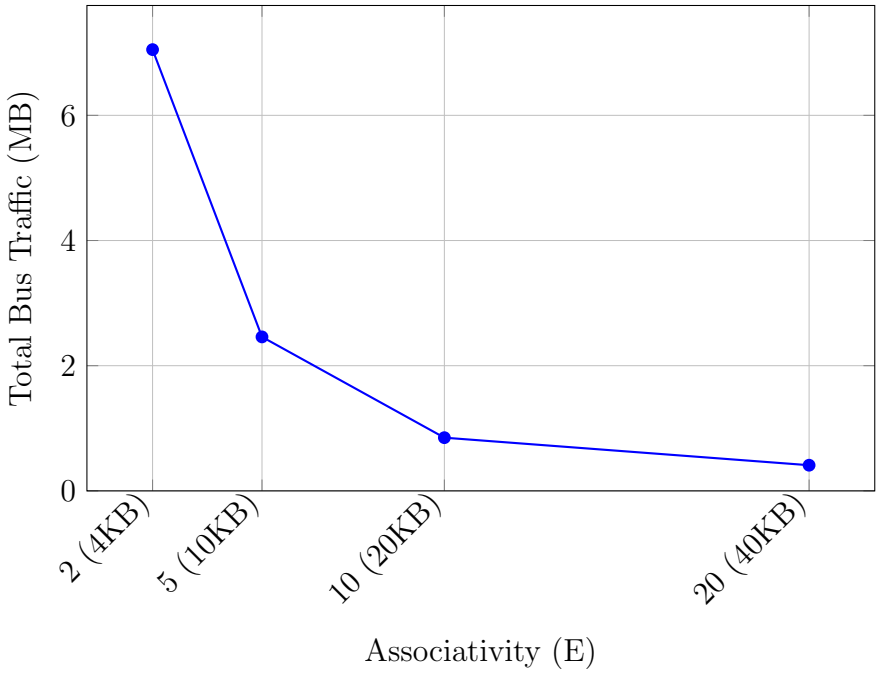

Figure 13: Total bus traffic vs. associativity (E)

3.9 Key Observations on Associativity Impact

1. **Diminishing Returns**: While increasing associativity from 2 to 20 reduces miss rates by approximately 95%, the most significant improvements occur in the initial increases (from 2 to 5), with diminishing returns as associativity increases further.

2. **Execution Time Improvement**: Execution cycles decrease by approximately 58% when moving from E=2 to E=20, with the most substantial gains (about 36%) achieved by increasing from E=2 to E=5.

3. **Bus Traffic Reduction**: Total bus traffic decreases from around 7MB at E=2 to just 0.4MB at E=20, a reduction of approximately 94%.

4. **Conflict Misses**: The significant reduction in miss rate with higher associativity indicates that conflict misses were a dominant factor in cache performance, particularly for Core 2 which consistently shows higher miss rates than other cores.

5. **Cost vs. Performance**: The data suggests that an associativity of 5 offers a good balance between performance and implementation cost, providing about 60% of the potential performance gains at 25% of the maximum tested cache size.

These results demonstrate that while increasing associativity consistently improves cache performance, the benefits follow a law of diminishing returns. The sweet spot appears to be around E=5 to E=10, where most conflict misses are eliminated without requiring excessive hardware resources.

3.10 Impact of Block Size on Performance

To complement our analysis, we studied the effects of varying the block size by changing the block bits (b) from 3 to 5, which corresponds to block sizes of 8, 16, and 32 bytes. This experiment was conducted while maintaining a constant set index of 6 bits and associativity of 2.

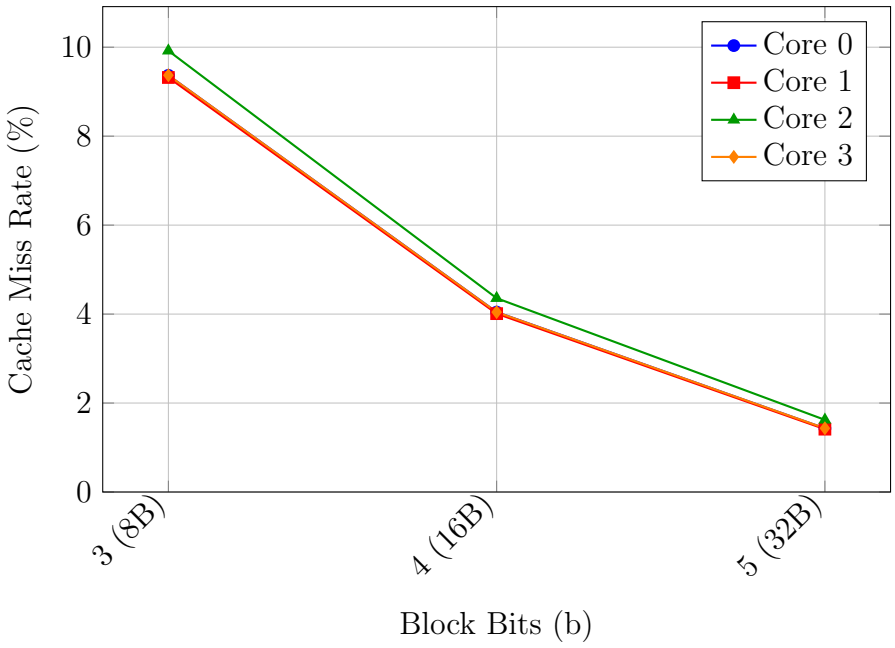

Figure 14: Cache miss rates vs. block bits (b) across different cores

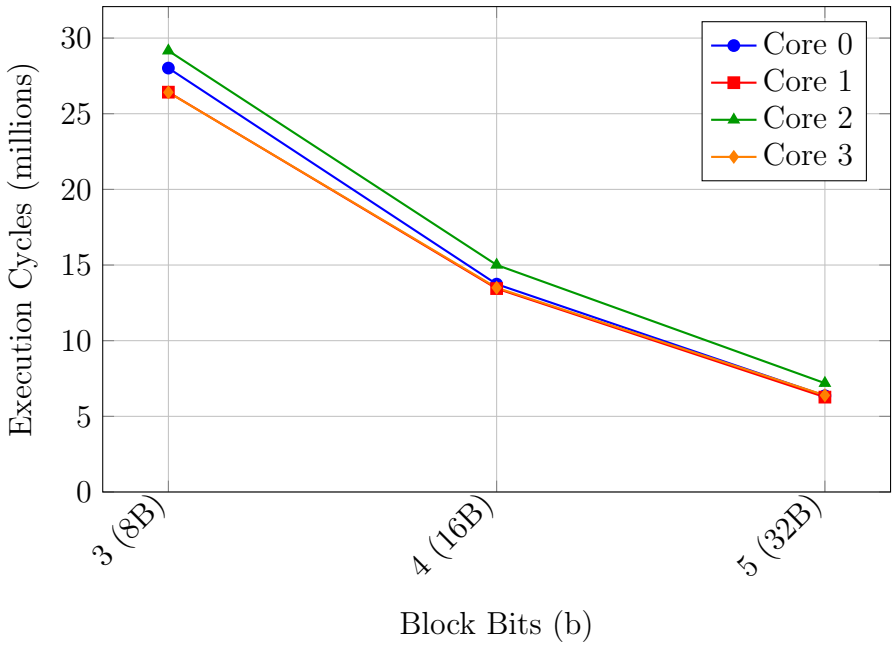

Figure 15: Execution cycles vs. block bits (b) across different cores

3.11 Bus Traffic Analysis with Varying Block Size

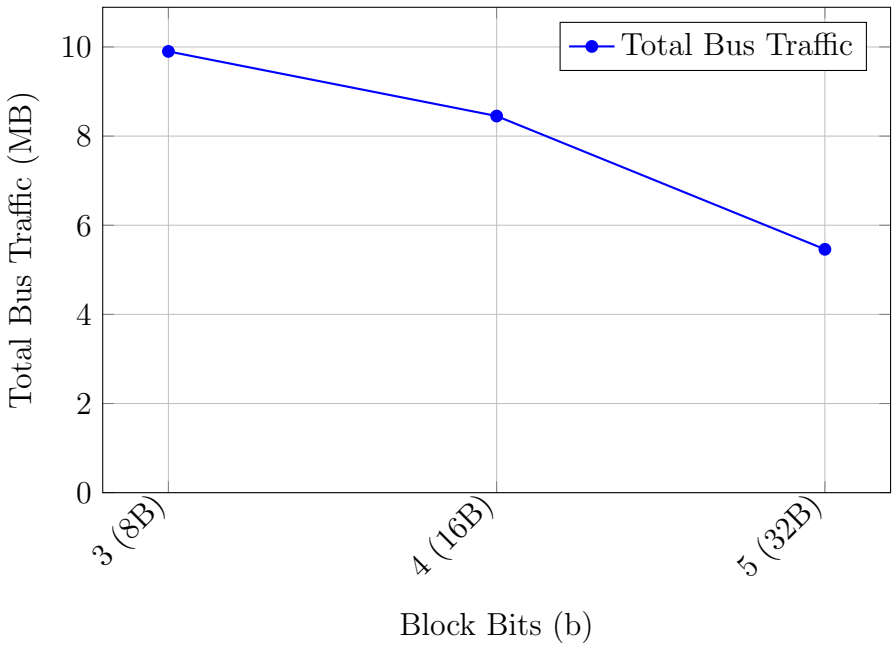

Figure 16: Total bus traffic vs. block bits (b)

3.12 Key Observations on Block Size Impact

1. **Miss Rate Improvements**: Doubling the block size from 8B to 16B reduces the miss rate by approximately 57%, and doubling again from 16B to 32B further reduces it by about 65%. This demonstrates the effect of spatial locality.

2. **Execution Time Reduction**: The execution time decreases by approximately 51% when moving from b=3 to b=4, and by another 54% when moving from b=4 to b=5, showing significant performance benefits with larger block sizes.

3. **Bus Traffic Efficiency**: While larger blocks transfer more data per miss, the overall bus traffic decreases with increasing block size due to the substantial reduction in miss rate, demonstrating better bandwidth utilization.

4. **Exploitation of Spatial Locality**: The dramatic performance improvements with larger block sizes indicate strong spatial locality in the application's memory access patterns.

5. **Diminishing Returns**: While we observe significant improvements across the tested range, we would expect diminishing returns with further increases in block size due to increased latency and potential for cache pollution.

These results highlight the importance of selecting an appropriate block size that balances the exploitation of spatial locality with the efficient use of cache capacity and memory bandwidth. For this application, a 32-byte block size (b=5) provides excellent performance compared to smaller block sizes.